

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int rank, size, N = 16;
    int array[N], local_sum = 0, total_sum = 0;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int elements_per_proc = N / size;
    int *sub_array = (int*)malloc(elements_per_proc * sizeof(int));

    if (rank == 0) {
        // Initialize the array with values
        for (int i = 0; i < N; i++) {
            array[i] = i + 1; // Example: [1,2,3,...,16]
        }
    }

    // Scatter the array to all processes
    MPI_Scatter(array, elements_per_proc, MPI_INT, sub_array, elements_per_proc,
MPI_INT, 0, MPI_COMM_WORLD);

    // Compute local sum
    for (int i = 0; i < elements_per_proc; i++) {
        local_sum += sub_array[i];
    }

    // Display intermediate sum at each processor
    printf("Processor %d - Partial Sum: %d\n", rank, local_sum);

    // Reduce all partial sums to the total sum at root (rank 0)
    MPI_Reduce(&local_sum, &total_sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Total Sum: %d\n", total_sum);
    }

    free(sub_array);
    MPI_Finalize();
    return 0;
}

```

-----

-----